

The Mist



A 2D top-down RPG set in a corrupted forest where the player fights through turn-based combat and procedurally generated dungeons to uncover and destroy the source of a spreading mist.

Brona Keevers-Roux
Course: Computer Science
Supervisor: Brendan Lyng
Target Platform: Windows

Contents

1. Introduction	4
2. Final Game Overview	6
Core Mechanics	6
Party System	6
Dual Gameplay - Hamster Companion	6
Turn-Based Combat System	7
Procedural Generation System	7
Progression Systems	8
Inventory	8
Controls	8
Intended Player Experience	8
Target Platform	9
3. Implemented Design & Game Systems	12
Core Gameplay Loop	12
Player Mechanics	13
Enemy and AI Systems	14
Procedural Level Design	15
Progression, Upgrades, and Difficulty	17
System Integration Summary	18
4. Production Process & Project Management	19
Development Approach	19
Sprint Structure and Milestones	19
Tools and Development Pipeline	20
What Worked Well	22
Challenges and Changes During Development	22
Summary of Process	23
5. Playtesting & Iteration	24
Playtesting Approach	24
Playtest Setup	24
Playtest 1 Overview	25
Key Issues Identified	26
Changes Made After Playtesting	27
Iteration Process and Development Impact	29
Conclusion	29
6. Technical Challenges & Solutions	30

Enemy AI Behaviour and Decision Consistency	30
Procedural Generation Reliability and Layout Stability.....	31
Input System Conflicts and State Management.....	32
Dynamic Shader for Party-Based Highlighting	32
Summary of Technical Challenges.....	33
7. Reflection & Learning	34
Technical and Programming Skills	34
Design Lessons Learned	34
Production and Time Management	35
Semester 1 Research into Practice.....	35
What I Would Do Differently.....	36
Overall Reflection	36
8. Future Work & Unimplemented Features	37
Crafting System and Harvestable Resources	37
Additional Enemy Types and Status Effects	37
Localisation and Language Support	37
Puzzle-Based Temple Progression.....	38
Expanded Skill Tree	38
Expanded Quest System.....	38
Summary	39
9. Conclusion	40
10. Appendices.....	41
11. Credits	41
Audio	41
Assets	41
Tools	41

1. Introduction

This report presents the development and evaluation of *The Mist*, a 2D top-down role-playing game created as my final year game development project. The game was developed as a solo project using the Unity engine for Windows PC. *The Mist* combines exploration, turn-based combat, progression systems, procedural generation, and dialogue within a fantasy setting. The player takes control of a protagonist travelling through a corrupted forest where a dark mist has caused the local wildlife to become aggressive. By exploring dangerous areas, battling enemies, interacting with NPCs, and progressing through procedurally generated temples, the player gradually becomes stronger while working towards uncovering and stopping the source of the corruption.

The original concept for the project included a wider range of planned features such as puzzle-solving mechanics, multiple elemental abilities, and dual gameplay sections involving both the main character and a companion hamster. As development progressed, the project scope was refined in order to focus on the strongest core systems and ensure the final game remained cohesive and complete. Rather than attempting to implement every planned feature, priority was given to combat, exploration, progression, dialogue, and procedural temple generation. This allowed the project to maintain a clear gameplay loop while remaining achievable within the available time.

The main aim of the project was to create a complete and functional game that demonstrates technical complexity across a range of systems. These included enemy AI, procedural generation, UI systems, inventory management, save and load functionality, and overall project structure. Another key aim was to create a game that offers replayability through randomly generated temple layouts and strategic turn-based combat encounters. Alongside the technical goals, I also wanted the game to provide an enjoyable player experience centred around progression, exploration, and a sense of adventure.

Several constraints influenced development throughout the project. As the game was created by a single developer, all programming, design, testing, balancing, and implementation had to be completed independently. This required careful scope management and prioritisation of features. Time was also an important constraint, as development needed to be completed alongside academic deadlines while allowing enough time for testing and refinement. While Unity is well suited to 2D development, the combination of systems such as combat, dialogue, procedural generation, inventory, and saving meant the project had to be carefully structured to keep complexity manageable as it grew. Asset creation was another limitation, as producing all art, animation, and audio independently would have been highly time consuming. Some visual assets were created personally, while the game's original music and sound effects were produced in collaboration with a music student. Because of this, development time was focused primarily on gameplay systems, with some visual elements kept simple where necessary.

Despite these constraints, the final version of *The Mist* became a playable and complete game containing a range of implemented systems. The finished project includes world

exploration, turn-based combat with reactive enemy behaviour, a party member system, levelling with stat progression and a skill tree, inventory and equipment systems, NPC dialogue, dual gameplay sections involving the hamster companion for small platforming sections, procedurally generated dungeon rooms using a grid-based layout system inspired by titles such as *The Binding of Isaac*, and save/load persistence. Together, these systems create a cohesive gameplay experience that reflects the original concept while adapting realistically during production.

This report will evaluate both the final game and the development process used to create it. Section 2 provides an overview of the completed game and its implemented features. Section 3 examines the core systems implemented in the final build, including gameplay mechanics, combat, progression, and procedural generation. Section 4 discusses the production process, project management methods, and development pipeline used throughout the project. Section 5 focuses on playtesting, feedback, and how the game was improved through iteration. Section 6 outlines the main technical challenges encountered during development and the solutions used to overcome them. Section 7 reflects on the skills developed, lessons learned, and how earlier research informed the final outcome. Section 8 discusses future work and features that were planned but not completed. Finally, Section 9 concludes the report by evaluating the overall success of the project and summarising the key outcomes of development.

Overall, this report aims to demonstrate that *The Mist* is both a complete game project and a substantial technical undertaking that required planning, iteration, and problem solving throughout development.

2. Final Game Overview

The Mist is a 2D top-down role-playing game (RPG) featuring turn-based combat, exploration, and light narrative progression. The game is set within a corrupted fantasy forest, where a mysterious dark mist has spread across the land, infecting wildlife and turning them hostile. The player takes on the role of a young adventurer, Erin, accompanied by their pet hamster, Milo, as they travel through these dangerous environments, clear corruption, and progress through a series of procedurally generated dungeon areas. Along the journey, the player becomes progressively stronger through combat, unlocks new abilities, recruits party members, and gradually works towards uncovering and defeating the source of the corruption.

Core Mechanics

The player moves between connected areas containing enemies, NPCs, shops and entrances to procedurally generated temple sections. Exploration allows the player to find resources, trigger encounters, and discover optional rewards that support progression.

Party System

A key feature of the game is the party system. As the player progresses, additional characters are recruited and become part of the active party in combat. Each party member can equip a weapon, armour piece, and scroll, which provide stat bonuses and influence combat effectiveness.

Each party member contributes differently depending on attributes and equipment, allowing the player to develop varied strategies across the team. Equipment improves core stats such as health and attack while also expanding available actions through weapons and scrolls that unlock additional abilities.

This creates distinct combat roles within the party and supports different playstyles without requiring direct control switching between characters.

Dual Gameplay - Hamster Companion

Alongside the main character, the game includes a dual gameplay mechanic involving a hamster companion. The main game is presented from a top-down perspective, while the hamster sections switch to a side-scrolling platforming view focused on navigating small cave areas.

These sections take place in compact cave environments accessible only to the hamster, allowing it to retrieve items, activate mechanisms, or unlock progression paths for the main character.

Turn-Based Combat System

Combat is structured around turn-based decision-making where both the player and enemies act in alternating phases. The player selects actions for any party member during their turn, with each character acting once per full turn cycle before becoming unavailable again until all members have acted. This promotes balanced use of the full party rather than reliance on a single character.

Enemy behaviour is reactive, with decisions influenced by factors such as player health and prior actions. Enemies also feature elemental weaknesses and resistances, encouraging strategic adaptation. Key enemy information is not fully visible by default, promoting observation and progression-based understanding.

Skill tree upgrades expand combat capability over time, including extended timing windows for perfect attacks and blocks, increased ultimate charge capacity, and improved enemy information visibility.

Combat includes multiple action types such as single-target attacks, area abilities, and healing skills. Mana is used for skills, while basic attacks regenerate it, creating a consistent resource loop.

An Ultimate Attack system allows multiple party members to combine into a single high-impact action. Ultimate charge is built through consistent performance in combat, by successfully executing perfect attacks and blocks, reinforcing skill-based engagement.

Consumable items provide additional tactical support during encounters, allowing the player to restore health, recover mana, or revive downed party members. Party health persists across battles and is only restored through levelling or resting, increasing the importance of long-term resource management.

If the player is defeated, the encounter is reset. Health is restored to its starting state and consumables used are returned, preventing progression loss from difficult encounters.

Procedural Generation System

Temples are generated using a grid-based system. A base layout is first created from a central spawn room, forming a connected structure of rooms. Once generated, room types are assigned through a post-processing step. The furthest rooms from the spawn point are replaced with relic and boss rooms, ensuring structured progression through exploration.

Remaining rooms are randomly assigned as treasure rooms, excluding the spawn room. Relic rooms must all be activated before access to the boss room is unlocked.

Each temple must meet minimum structural requirements. If a layout fails validation (such as insufficient rooms or invalid structure), it is discarded and regenerated.

Completing a temple rewards the player with an elemental ability such as fire or ice, which is used in the main world to unlock new paths and interact with environmental obstacles.

Progression Systems

Progression is handled through levelling, equipment, and inventory systems. Defeating enemies grants experience points, with each party member levelling independently and earning attribute points that can be allocated separately, enabling distinct builds such as tank or damage-focused roles.

Quests are split into main and side objectives and can be viewed through the player menu. Completing quests rewards skill orbs, which are used to unlock nodes in the skill tree. Equipment can be found or purchased and provides stat bonuses while influencing combat style through different ability unlocks and role specialisation.

Inventory

The inventory system manages equipment, consumables, quest items, and treasure. Equipment is handled outside of combat and can be equipped or changed to adjust character stats and abilities. Consumables are restricted to combat usage and include healing and revival items. Treasure items have no gameplay function beyond being sold for gold, while quest items support progression objectives.

Controls

The game is designed for keyboard and mouse input on PC. Movement uses WASD or arrow keys, sprint is mapped to Shift, and interactions use the E key. The player menu is accessed via Tab and the pause menu via Escape. Dialogue can be progressed using Enter or mouse click.

All menu navigation, including player menus, shops, and combat interfaces, can be controlled using either keyboard or mouse input, allowing flexible and accessible interaction throughout the game.

Intended Player Experience

The intended experience of *The Mist* centres on exploration, progression, and strategic decision-making. The player gradually becomes stronger through new abilities, equipment, and party development.

Exploration is intended to feel rewarding, with optional encounters, hidden rewards, and procedural variation encouraging replayability. Combat is deliberately methodical rather than fast-paced, requiring the player to consider resource usage and enemy behaviour when choosing actions.

Overall, the experience is designed to balance challenge and progression, giving the player a consistent sense of advancement as they move deeper into increasingly dangerous areas of the world.

Target Platform

The Mist is developed for Windows PC, with keyboard and mouse as the primary input method. The game is designed to run as a standalone executable.

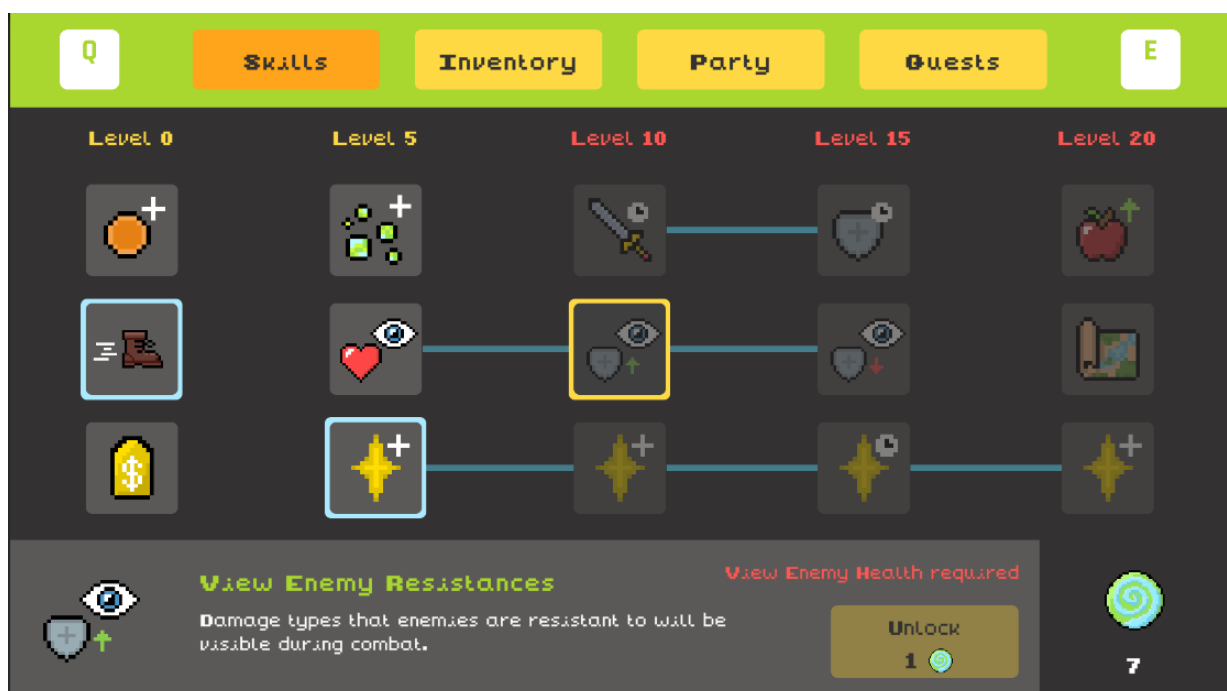
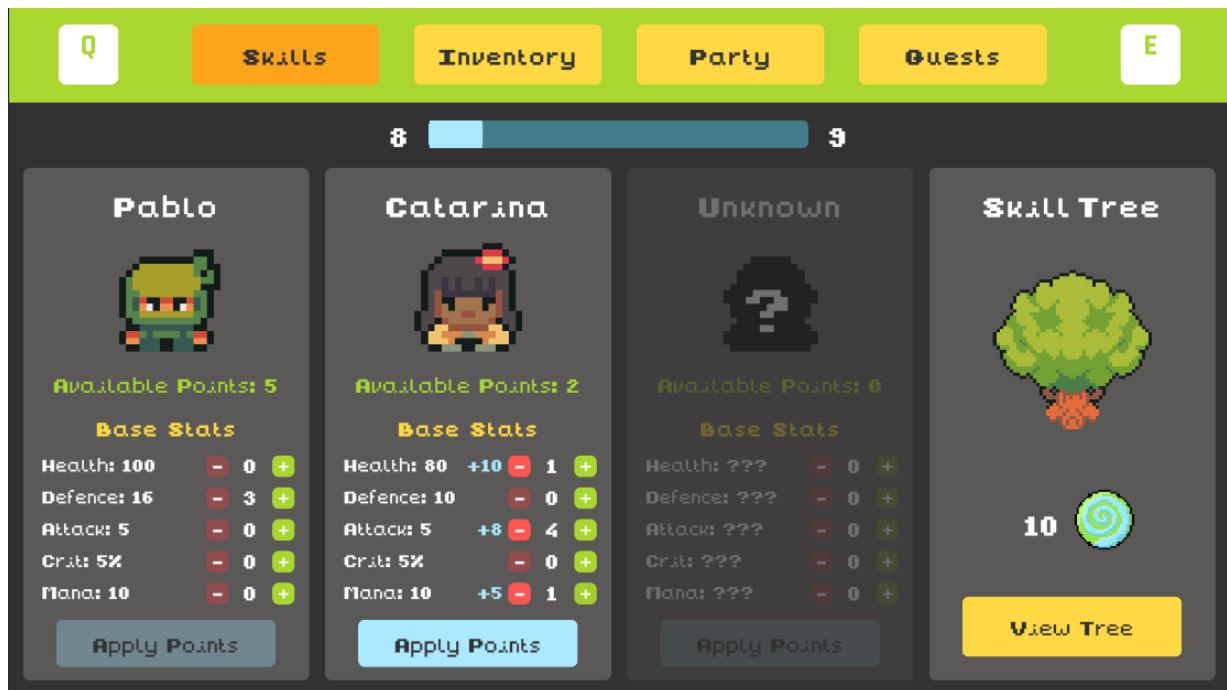
Figure 1: Screenshot of gameplay with brief annotations

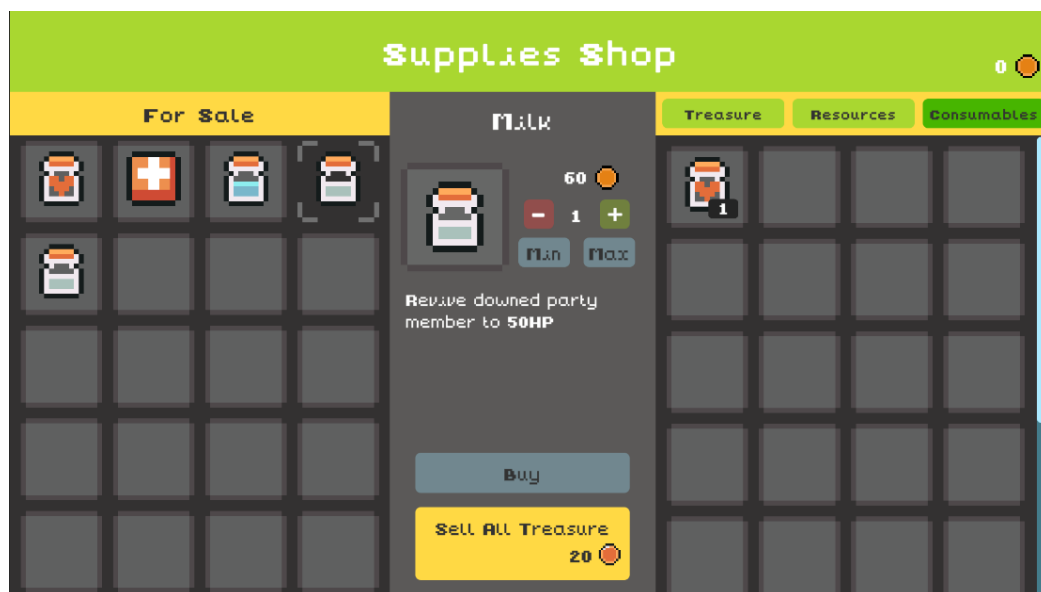
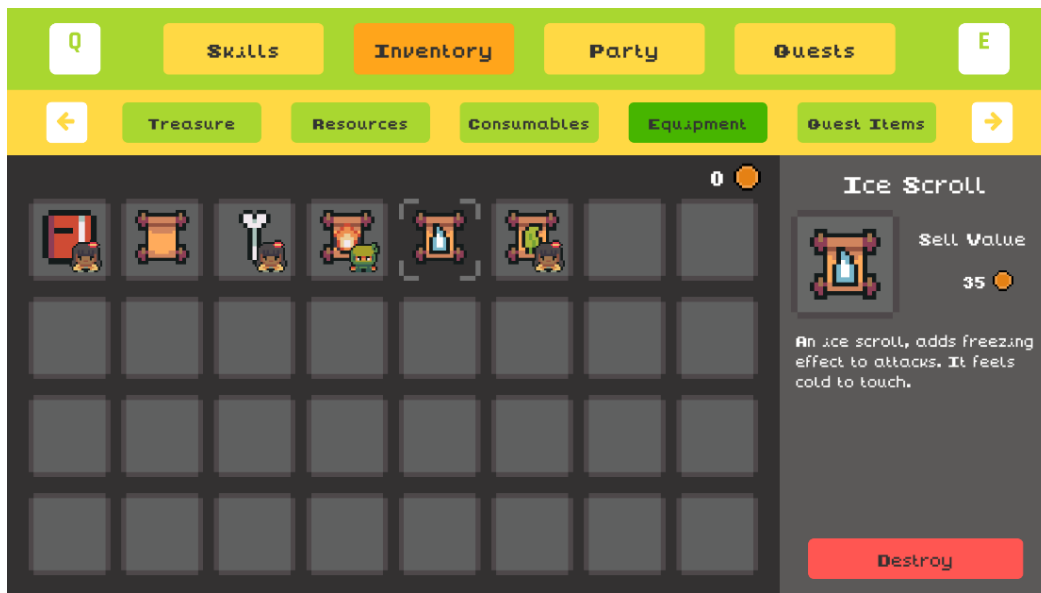
Annotated gameplay screenshot showing core combat elements including player party, enemies, and UI components during a typical encounter.



Figure 2: UI or screen flow (menu gameplay progression)

Player Menus and shop





3. Implemented Design & Game Systems

This section outlines the implemented systems in *The Mist*, focusing on how each system functions, how they interact, and key design decisions made during development.

Core Gameplay Loop

The core gameplay loop in *The Mist* centres on exploration, combat, and player progression. Players move through the main world completing quests, interacting with NPCs, visiting shops, and locating temple entrances. These areas provide opportunities to prepare for future encounters through equipment upgrades, resource management, and party development before progressing into more dangerous temple sections.

Inside temples, gameplay shifts into a procedural room-based structure. Players traverse connected rooms containing combat encounters, treasure, relic objectives, and branching paths leading toward the boss room. This creates a balance between direct progression and optional exploration.

Combat drives progression through experience and resources. Because health persists across encounters, damage accumulation becomes a key strategic factor during longer temple runs.

Temple completion provides major progression milestones, rewarding elemental abilities used to unlock new areas in the main world.

Figure 3: Core gameplay loop diagram

Showing the interaction between moment-to-moment gameplay and long-term temple progression.

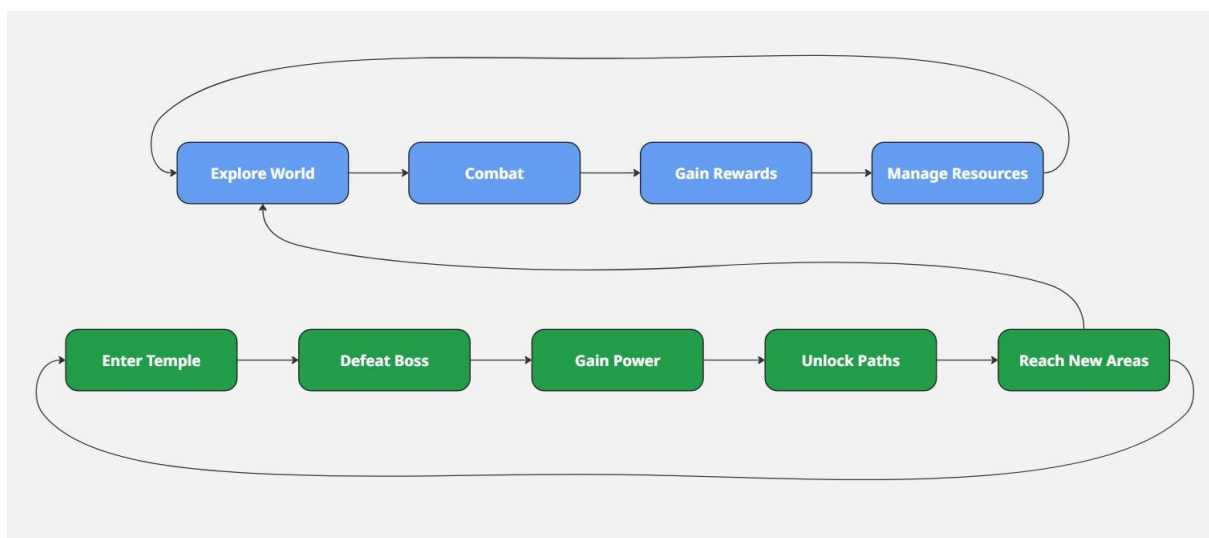


Table 1: Core Gameplay Loop Breakdown

Summarises the main stages of gameplay in The Mist, outlining player actions, associated systems, and the resulting outcomes within the core game loop.

Stage	Player Activity	Systems Involved	Outcome
Exploration	Move through the main world and temples	Movement system, input system, minimap	Discovery of objectives, enemies, and resources
Preparation	Interact with NPCs, shops, inventory	UI system, inventory, equipment, progression	Improved readiness for combat encounters
Combat	Turn-based battles with enemies	AI system, combat system, skills, items	XP gain, loot, resource changes
Progression	Level up and unlock abilities	Skill tree, levelling system, attributes	Increased combat capability
Temple Completion	Defeat boss and clear temple	Procedural generation, boss AI	Unlock elemental ability / game progression

Player Mechanics

Player control is implemented through a top-down movement system using keyboard input, with contextual interactions handled through dedicated input actions. The system is built using Unity's Input System, with actions managed through an Input Actions Asset. Different action maps are enabled or disabled depending on the current game state. This ensures inputs remain consistent across the game while preventing conflicts between gameplay systems.

The player can sprint, interact with world objects, and navigate UI systems such as inventory, shops, and menus using either keyboard or mouse input. Interaction systems are context-sensitive, meaning the same input is reused across different gameplay states including exploration, dialogue, and UI navigation. This reduces input complexity while maintaining consistency across all systems.

A minimap system supports navigation in both the main world and temples, updating in real time to reflect explored areas.

A multi character party system is implemented, where the player controls all party members, with each character participating during combat. Outside combat, unlocked party members follow the player in the main world, reinforcing their presence and supporting immersion.

Enemy and AI Systems

Enemy behaviour in *The Mist* is split between two connected systems: overworld navigation AI and turn-based combat decision-making.

In the main world, enemies use a custom A* pathfinding system to navigate environments and move between patrol states. This allows them to wander naturally through areas while avoiding obstacles and following valid movement paths. A finite state machine governs their behaviour, with states including wandering, patrolling, chasing, and combat engagement.

When the player enters an enemy's detection range, the enemy transitions into a chase state and uses A* pathfinding to pursue the player directly. If an enemy reaches the player, a combat encounter is triggered, pulling all enemies in the area into a single battle instance. If the player exits detection range beforehand, enemies return to wandering states.

Enemies are spawned dynamically at runtime within each area. This introduces variation in both the number and type of enemies present during exploration, meaning that each traversal of an area can present slightly different encounter compositions.

Once combat begins, enemy behaviour transitions into a turn-based decision system based on a points-based evaluation model. Each enemy evaluates all available actions each turn and assigns them a score based on the current combat state. The action with the highest score is selected and executed.

This system functions as a utility-based AI, where actions are scored using weighted conditions such as player health, enemy health, party composition, positioning, and prior actions. Healing is prioritised at low health thresholds, while offensive actions are favoured when the player is vulnerable or exposed.

To prevent the AI from always selecting the mathematically optimal action, a controlled element of randomness is applied to the scoring process. This ensures that enemy behaviour remains varied and prevents encounters from feeling overly deterministic or unfair, while still maintaining strategic consistency.

Enemies also feature elemental weaknesses and resistances, which affect how incoming damage is calculated. These attributes influence how effective player attacks are against different enemy types, encouraging adaptation in combat strategy.

Each enemy is implemented using a modular, data-driven structure defining attributes such as health, damage values, elemental affinity, and available actions. This allows for efficient creation and balancing of different enemy types while maintaining consistent underlying AI behaviour. Each enemy type is assigned a specific set of abilities, which may include direct attacks, area-based damage, or status effects. These actions are evaluated each turn through the scoring system, ensuring behaviour is driven by combat state rather than fixed patterns.

The combat system is structured around turn phases, where the player and enemies take actions in separate stages. During the enemy phase, all active enemies may act, allowing multiple actions to occur within a single turn cycle. This can increase encounter pressure in fights involving multiple enemies, as several actions may be resolved before the player's next turn.

Overall, the enemy AI system combines state-driven pathfinding in the main world with a points-based combat decision model and controlled randomness. This creates enemies that feel reactive during exploration and strategically adaptive during combat, supporting both environmental tension and tactical gameplay depth.

Procedural Level Design

Dungeon environments are generated using a grid-based procedural system. Each temple begins by generating a base layout of connected rooms branching outward from a central spawn room. This initial structure forms a fully traversable graph of basic rooms, ensuring all areas are reachable before specialised room assignment takes place.

Once the base layout is generated, room types are assigned through a post-processing step. The furthest rooms from the spawn point are identified and replaced with key progression rooms, specifically relic rooms and the boss room. This ensures that major objectives are positioned at the end of exploration paths, encouraging full traversal of the dungeon structure.

Treasure rooms are then assigned by randomly replacing remaining eligible rooms within the layout. The spawn room is excluded from this process to preserve player starting conditions. This introduces variation in reward placement while maintaining a consistent overall dungeon structure.

Room types include:

- Combat rooms (enemy encounters)
- Treasure rooms (reward chests)
- Relic rooms (progression objectives)
- Boss room (final encounter)

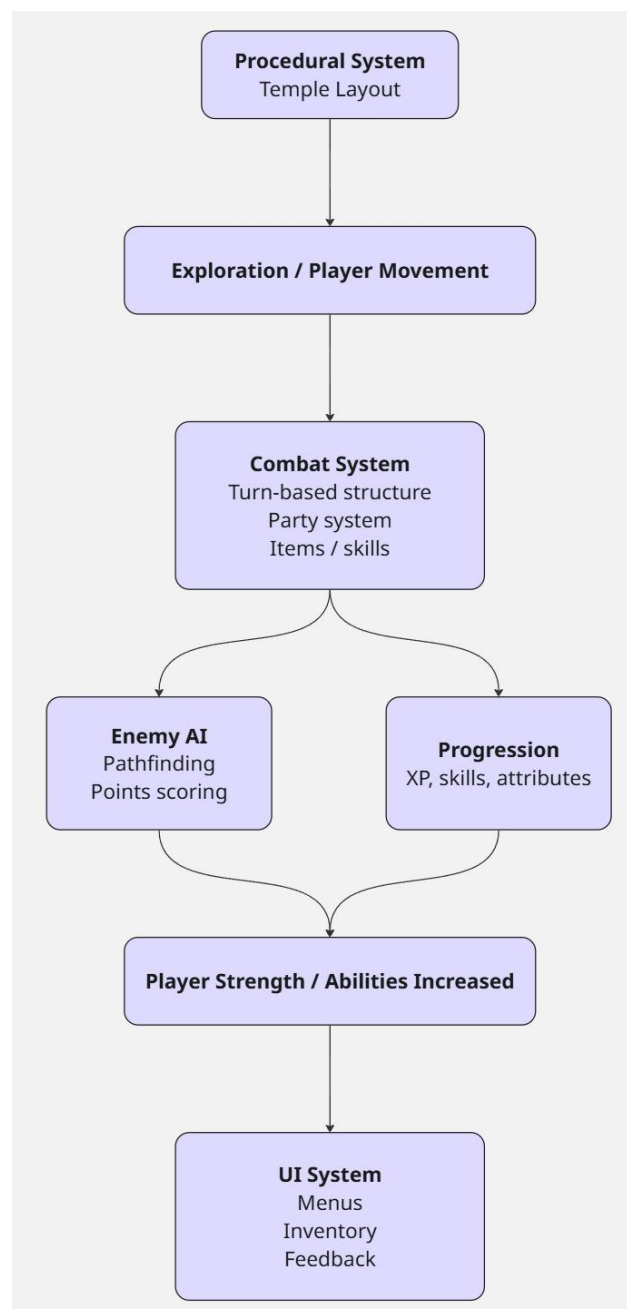
Relic rooms are positioned at distant points within the layout and must all be activated before access to the boss room is unlocked. This enforces structured progression through the procedural space, ensuring that players cannot bypass intended gameplay flow. Treasure rooms provide additional rewards such as items or resources. These rooms introduce risk-reward decision-making, as players must choose whether to explore beyond the main progression path in exchange for potential advantages.

A key design principle of the system is that procedural generation supports structured gameplay rather than relying on randomness alone. Although layouts vary between runs, each dungeon maintains a consistent progression arc from exploration to objective completion and final encounter.

A key development decision was the adoption of this staged generation approach (base layout → key room placement → reward distribution). This improved reliability and ensured all generated layouts remain solvable, while reducing the risk of disconnected or poorly structured room configurations. If a generated temple fails to meet minimum structural requirements, such as a sufficient number of rooms or valid end-point placements, the entire layout is discarded and regenerated from scratch.

Figure 4: High-level system interaction diagram

Illustrates how core gameplay systems in The Mist interact, including procedural generation, exploration, combat, enemy AI, progression, and UI.



Progression, Upgrades, and Difficulty

Progression in *The Mist* is driven through multiple interconnected systems, including experience-based levelling, attribute allocation, skill tree upgrades, and equipment improvements.

Each party member levels up independently and gains attribute points that can be allocated separately. This allows for distinct build specialisation across the party.

The skill tree system provides combat-focused upgrades that gradually expand the player's tactical options. These include extended timing windows for perfect attacks and blocks, increased ultimate charge capacity and the ability to reveal enemy information such as health, weaknesses, and resistances.

A core mechanic of the combat system is the perfect timing system, where well-timed inputs during attacks or blocks reward increased damage output or reduced incoming damage. This introduces a skill-based layer to combat that operates alongside turn-based decision-making.

Equipment further supports progression by providing stat bonuses and influencing overall combat effectiveness. Weapons and scrolls also expand each character's available actions, unlocking additional abilities and attack options that define their combat role. This contributes to build variation by reinforcing different playstyles across party members, allowing characters to be specialised through both attributes and available actions.

Difficulty is shaped through resource persistence and encounter structure rather than explicit scaling systems. Party health carries across encounters within temples, meaning sustained damage has long-term consequences and increases tension over extended exploration. However, consumable item usage is partially mitigated through a recovery system where items used in a failed encounter are restored upon defeat. This prevents soft-lock scenarios while still maintaining the strategic importance of resource management during combat.

As progression continues, encounter difficulty increases through the introduction of more challenging enemy types and a higher frequency of multi-enemy engagements, requiring the player to manage multiple threats simultaneously and make more efficient use of party actions and resources.

System Integration Summary

A key focus of development was ensuring that all gameplay systems interact cohesively rather than functioning in isolation. Combat contributes directly to progression through experience gain and resource rewards, supporting overall character development. In turn, progression systems feed back into combat by unlocking new abilities, improving core attributes, and expanding the tactical options available to the player.

Procedurally generated temples provide the structural framework that connects these systems, offering varied environments in which combat encounters, exploration, and progression mechanics operate simultaneously. This ensures that each gameplay loop remains consistent while still allowing for variation in layout and encounter composition.

This interconnected structure was a deliberate design choice to ensure that all systems reinforce one another, creating a consistent cycle of exploration, combat, and character development rather than disconnected gameplay features.

During development, a key consideration was maintaining a cohesive gameplay structure after simplifying early design ideas. Planned mechanics such as puzzle-based progression were reduced in scope to ensure focus remained on the core systems of exploration, combat, and progression. As a result, existing mechanics were refined and expanded to maintain depth and ensure a consistent gameplay experience.

Table 2: Implemented Feature Summary

Provides an overview of the core gameplay systems implemented in The Mist, summarising their function and role within the overall gameplay experience.

System	Description	Key Role in Gameplay
Movement System	Top-down player control using Unity Input System	Exploration and navigation
Combat System	Turn-based party combat with timing mechanics	Core gameplay loop
Enemy AI	Utility-based decision-making and A* pathfinding with finite state machine	Challenge and encounter design
Procedural Generation	Grid-based dungeon generation system	Replayable temple layouts
Progression System	Levels, attributes, skill tree, quests	Player growth and scaling
Inventory System	Items, equipment, consumables	Resource and combat support
UI System	Menus, interaction, HUD	Player feedback and control

4. Production Process & Project Management

This section explains how *The Mist* was planned, developed, and refined during Semester 2. It covers the development approach, sprint structure, tools used, and how the project changed over time. Rather than following a strict production methodology, the project used a flexible iterative workflow that evolved as systems were built and tested.

Development Approach

The project was developed using an iterative approach inspired by Scrum, although adapted for solo development. Instead of formal Scrum roles or meetings, work was split into short development cycles where one or two systems were focused on at a time.

Each cycle followed a consistent pattern: a feature was implemented, tested directly in Unity, and then refined based on how it behaved in practice. This was particularly important because many of the systems in *The Mist* are interconnected. Combat, for example, only made sense when tested alongside enemy AI, progression, and consumables, so these systems were integrated earlier rather than being developed in isolation.

A key principle throughout development was maintaining a continuously playable build of the game. Even when systems were incomplete or basic, the core loop of movement, combat, enemies, and progression was always functional. This allowed issues to be identified early and prevented major integration problems later in development.

As the project progressed, it became clear that not all planned features were necessary to achieve the intended experience. Instead of attempting to implement every planned mechanic, development shifted toward strengthening the core loop of exploration, combat, and progression. This helped ensure the final game felt cohesive rather than fragmented across too many systems.

Sprint Structure and Milestones

Development was organised into informal sprint cycles, with each cycle focused on delivering and integrating a specific system or set of related features. Rather than following a strict Scrum framework, these sprints were used as a practical way to break development into manageable phases and ensure steady progress across all core systems.

Each sprint typically involved planning tasks in Trello, implementing the required features in Unity, and then testing them directly in-game. Once a system was functional, it was refined and adjusted based on how it interacted with existing mechanics. This was important because many systems in *The Mist* are interconnected, meaning that changes in one area often affected others.

The milestones below summarise the main development phases completed during Semester 2. While these phases are presented in a linear order, in practice there was overlap between sprints as systems were revisited, balanced, and integrated over time.

Table 3: Sprint / Milestone Summary

Outlines the main development sprints completed during Semester 2, including focus areas and key features implemented in each stage.

Sprint	Focus	Key Features Implemented
1	Hamster gameplay	Hamster holes, scene, camera, platformer controls
2	UI and their systems	Player menu, quests, inventory, equipment, party, shop, levelling, loading screens
3	Combat, enemies and skill tree	Enemy spawning, combat screen, combat systems, skill tree
4	Procedural generation	Grid system, room assignment, boss/relic logic
5	Chests, minimap	Chests, treasure rooms, music, minimap
6	Polish & balancing	UI improvements, AI tuning, bug fixing

This structure was not strictly linear, as some systems overlapped across multiple sprints. However, it provided a useful way to organise priorities and ensure that core gameplay features were completed early enough to allow proper testing and iteration.

Tools and Development Pipeline

A range of development tools were used throughout the project to support planning, implementation, and version control.

Table 4: Tool Usage Overview

Summarises the key development tools used throughout the project and their role in supporting design, implementation, and project management.

Tool	Purpose	How it was used in development
Unity	Game engine	Core implementation of gameplay systems
C#	Programming language	All gameplay logic and systems
Trello	Task management	Sprint tracking and feature organisation
Aseprite	Creating sprites	Used to create sprites for things such as all the skill tree icons
Miro	Visual planning	Gameplay loops, system diagrams, procedural flow
Git / GitHub	Version control	Code tracking, backups, iteration management

The development pipeline followed a consistent iterative structure. Features were initially planned using Trello and visualised in Miro where necessary, then implemented in Unity using C#. Once implemented, features were tested directly in Play Mode to evaluate behaviour in a real gameplay context.

Feedback from testing was used to debug, rebalance, or adjust systems before committing changes through GitHub. This cycle repeated continuously throughout development, with features often revisited multiple times rather than being treated as complete after initial implementation.

Figure 5: Development pipeline diagram

Illustrates the iterative development workflow used during production, showing how features moved from planning through implementation, testing, and iteration.

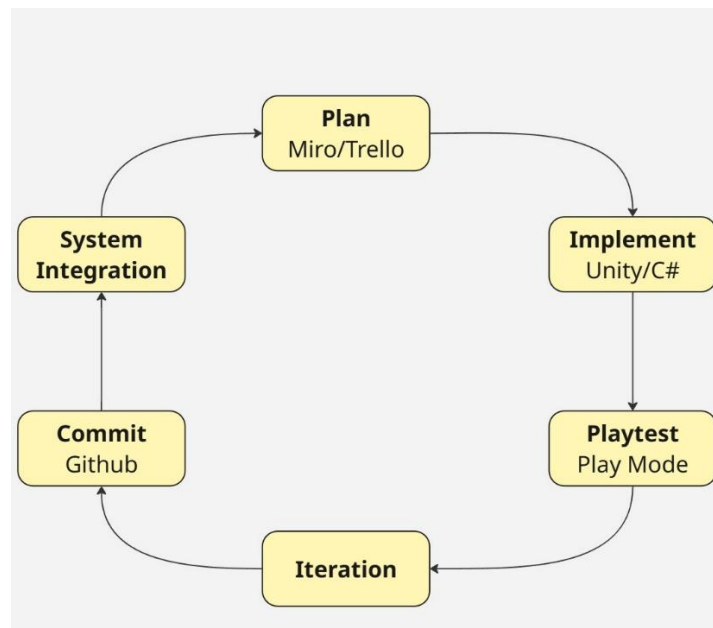
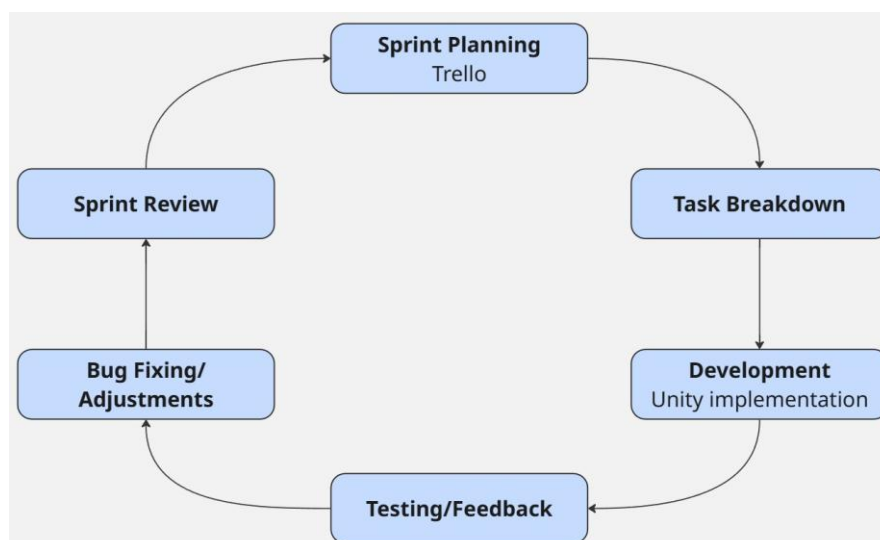


Figure 6: Sprint overview

Visual summary of the sprint structure used during development, outlining the progression of focus areas across Semester 2.



What Worked Well

One of the strongest parts of the development process was building systems early and testing them together. Instead of finishing one system completely before starting another, systems were gradually integrated. This meant issues were found earlier and could be fixed without major rework.

Another thing that worked well was keeping systems modular. Enemy AI, procedural generation, and progression were all built in a way that allowed them to be adjusted without rewriting large parts of the codebase.

Using short development cycles also helped keep progress steady. Each sprint had a clear focus, which made it easier to finish features without getting stuck trying to perfect everything at once.

Early prototyping of combat and movement also helped shape the rest of the game, since it became clear quite early what systems would actually work well together.

Challenges and Changes During Development

A few things changed during development once systems were actually built and tested in Unity.

One of the main changes was the removal of planned puzzle mechanics. These were initially intended as part of temple progression, but they were not implemented due to time constraints. Instead of attempting to include them in a reduced form, development focus was shifted toward improving the core gameplay loop. This ensured that combat, exploration, and progression systems were fully developed and cohesive.

The procedural generation system also changed quite a bit during development. I initially explored more complex generation approaches, but they were inconsistent and often produced layouts that didn't feel good to navigate. In the end, I moved to a simpler grid-based system because it gave me more control over the layout and produced cleaner, more readable temple structures. This also better matched the style of level design I was aiming for, where readability and navigation were more important than complexity.

Enemy AI was refined during testing into a points-based decision model with controlled randomness. Each enemy evaluates available actions and assigns them weighted scores based on the current combat state, with the highest scoring action being selected. This approach provides structured decision-making while still allowing variation in behaviour, preventing encounters from feeling overly deterministic.

The input system was also improved halfway through development. I split controls into different action maps for exploration, combat, and UI using Unity's Input System. This made a noticeable difference because it stopped inputs interfering with each other when switching between gameplay states.

Summary of Process

Overall, the production process for The Mist was iterative and adjusted throughout development. Initial plans provided a starting structure, but the final implementation was shaped by continuous testing, system refinement, and scope changes made during development. A Scrum-style approach helped keep work organised while still allowing flexibility when systems needed to change or be simplified.

Tools such as Trello, Unity, and Git were used throughout to manage tasks, implement features, and track versions of the project. This created a consistent workflow where features could be developed, tested, and revised without disrupting the overall project structure.

The main outcome of the process was successfully delivering a complete and playable game by prioritising core systems and adapting the scope where necessary. This reflects a practical development approach where iteration, prioritisation, and problem-solving were essential in moving the project forward.

5. Playtesting & Iteration

This section outlines how The Mist was tested during development and how feedback from playtesting informed iterative improvements across gameplay, UI, controls, and overall system stability. Testing was treated as an ongoing development process rather than a final-stage activity, with changes being implemented continuously as issues were identified through real gameplay sessions.

The primary goal of playtesting was to evaluate how well the core systems worked together in practice, particularly combat, UI navigation, movement feel, and interaction consistency. While systems may function correctly in isolation, playtesting helped highlight where mechanics broke down when combined or when used in unexpected ways by players.

Playtesting Approach

The approach to testing was iterative and informal but structured around specific development milestones. Testing was carried out at key points during development, usually after major systems or features were implemented or significantly modified.

Playtesting was conducted in three main methods:

- Self-testing during development in Unity
- Peer testing with other students
- A single observed in-person playtest session

Most feedback was collected after players completed their sessions, typically through written or verbal responses. In one in-person session, direct observation was also used to see how a player interacted with the game in real time. This combination helped identify usability issues that were not immediately obvious during solo development, particularly in areas where players hesitated or misunderstood interactions.

Rather than treating testing as a separate phase, it was embedded into the development cycle. Features were implemented, tested immediately in-engine, adjusted, and then retested. This created a continuous loop of refinement.

Playtest Setup

Testing was primarily conducted through a single structured playtest, supported by ongoing self-testing during development. The main playtest was carried out by distributing a build of the game to a small number of peers, who played it independently and provided feedback afterwards. This approach allowed players to experience the game without direct guidance, making their responses more reflective of a natural first-time playthrough.

In addition to this, one in-person session was conducted where a player tested the game while being observed. This provided more direct insight into how players interacted with systems in real time, particularly where players hesitated or became confused, which was not always captured in written feedback.

Table 5: Playtest Metadata

Summarises the context of playtesting sessions, including participants, environment, and testing focus.

Playtest	Participants	Location	Build Version	Focus Areas
Playtest 1	3 peers	Home / lab environment	Early Semester 2 build	Movement, UI navigation, interaction systems
Ongoing Testing	Solo (developer)	Unity Editor	Continuous builds	System integration, bug fixing, balancing

Playtest 1 Overview

The first structured playtest focused on an early but functional version of the core gameplay loop, including movement, basic combat, UI systems, and party interaction. At this stage, most core systems were implemented, but they had not yet been fully refined or balanced, making it a good point to evaluate how they performed together in practice.

The goal of this playtest was not to assess content depth, but to identify usability issues and determine whether the core systems felt clear and responsive from a player perspective. This included evaluating how intuitive movement felt, how easily players could navigate UI systems, and whether interactions between gameplay states (such as exploration, combat, and menus) were understood without explanation.

Because players were not given detailed instructions before testing, the session also helped highlight areas where the game did not communicate information clearly. This was particularly useful for identifying issues that would not typically appear during solo development, such as unclear controls, inconsistent input behaviour, or confusion when switching between systems.

Overall, this playtest acted as a baseline evaluation of the game's usability. It provided early insight into how players interpreted the game's mechanics, which informed the first round of iteration focused on improving clarity, responsiveness, and overall player experience.

Table 6: Playtest 1 Feedback and Iteration Summary

Summarises key feedback from the first structured playtest of The Mist and the corresponding changes implemented to improve gameplay feel, UI clarity, controls, and system stability.

Category	Feedback	Change Implemented
Gameplay Feel	Hamster jump feels too high / floaty	Increased jump force and gravity scale to improve responsiveness and reduce floatiness
UI / UX	New game warning appears unnecessarily	Added condition so warning only appears if an existing save file is detected
UI / Navigation	Tab menu uses Q, E, arrow images to show controls rather than being buttons themselves	Replaced image-based inputs with functional button-based UI elements
Controls	No mouse input for dialogue progression	Added left mouse click as an alternative input alongside Enter key
Inventory System	Inventory slots not keyboard navigable	Implemented WASD navigation for inventory grid selection
Party UI	Selected party member not clearly visible	Improved visual distinction for selected party member in UI
Bug Fix	Party interaction possible while following	Fixed issue by unselecting party member when they begin following state

Key Issues Identified

1. Movement Feel and Player Control

One of the main issues identified during testing was the feel of the hamster's movement, particularly jumping. Players described it as too floaty, which made platforming feel less precise than intended and reduced the sense of control.

Although the system was technically working, it didn't feel right in practice. This highlighted that getting movement "correct" isn't just about values behaving as expected, it also needs to feel responsive and grounded from the player's perspective. It became clear that physics needed to be tuned based on how it felt to play, not just whether it functioned.

2. UI Clarity and Input Consistency

Several issues came up around UI clarity, especially in how inputs were presented and handled. Some parts of the interface used static images instead of interactive elements, and input methods were not always consistent between systems.

This led to confusion, particularly in the tab menu and dialogue. Players weren't always sure what inputs were available, and in some cases expected mouse interaction where only keyboard input was supported. Even though the systems worked, the lack of clear feedback made them harder to use than intended.

3. Interaction and State Conflicts

A bug was also identified where players could still interact with party members while they were already in a following state. This pointed to a conflict between the interaction system and the party behaviour logic.

While it was a small issue on its own, it showed how easily inconsistencies can appear when systems overlap. If left unresolved, this kind of problem could lead to more noticeable issues elsewhere.

Overall Observations

Across all of the feedback, a clear pattern emerged: most issues weren't caused by missing features, but by how clearly the game communicated with the player.

Movement, for example, worked as intended but didn't feel responsive. UI systems were functional but didn't always make it obvious how to interact with them. In both cases, the problem wasn't the system itself, but how it was experienced.

This reinforced the importance of designing around player expectations. Systems need to be readable and intuitive, not just technically correct.

Changes Made After Playtesting

Following Playtest 1, a range of changes were made to improve overall clarity, responsiveness, and system stability. Most of these focused on refining how existing systems felt and behaved, rather than introducing new features.

Movement for the hamster was adjusted by increasing both jump force and gravity. This reduced the floaty feel and made platforming more responsive, giving players a better sense of control during traversal.

UI systems were also reworked to improve consistency. In the tab menu, image-based indicators were replaced with proper interactive buttons, making navigation more intuitive and closer to standard UI behaviour.

Dialogue interaction was expanded to include mouse input, allowing players to progress conversations using either the keyboard or a mouse click. This made interactions feel more natural and reduced friction during dialogue.

Inventory navigation was improved by adding WASD support, allowing players to move through item slots without relying on the mouse. This made the system more accessible and better suited to keyboard-focused play.

Clarity within the party UI was also addressed by making the selected character more visually distinct. This helped reduce confusion when managing the party or selecting targets during combat.

Finally, a bug related to party interaction was resolved. Party members could previously be interacted with while already following the player, which created inconsistent behaviour. This was fixed by ensuring characters are deselected once they enter the following state, preventing unintended interactions.

Figure 7: Before/after example of an iteration

Before and after comparison of party member selection UI, showing the selected character highlighted using a lighter colour while non-selected party members are darkened to improve clarity during selection.



Iteration Process and Development Impact

One of the main outcomes of playtesting was realising that most issues weren't caused by systems being broken, but by how they were presented to the player. In many cases, the underlying logic worked, but the way it was communicated made it harder to understand or use. This shifted the focus of development away from just getting systems working, towards making them clearer and easier to interact with.

Feedback wasn't treated as a list of isolated fixes. Instead, it was used to adjust how systems worked together as a whole. This was especially important for UI and controls, where even small inconsistencies had a noticeable effect on how the game felt to play.

Testing also showed the importance of seeing systems in context. Some problems only appeared when multiple systems were active at the same time, which isn't something that comes up during isolated development. This made it clear that regular in-game testing was just as important as building the systems themselves.

Conclusion

Playtesting played a key role in shaping The Mist into a more stable and usable experience. Early feedback highlighted issues with movement feel, UI clarity, and input consistency, which were addressed through targeted changes.

More importantly, it changed how development was approached. The focus moved from simply implementing systems to improving how those systems felt and how clearly they communicated with the player. Most of the improvements were small changes, but they had a noticeable impact on usability and overall game feel.

The iterative approach meant issues were picked up and resolved as they appeared, rather than building up over time. This helped keep the core gameplay loop stable while other systems were being developed.

Overall, the process showed that playtesting is not just about finding bugs, but about understanding how players experience the game. That insight was essential in making sure The Mist feels clear, responsive, and consistent to play.

6. Technical Challenges & Solutions

This section outlines the main technical challenges encountered during development of *The Mist*, focusing on enemy AI behaviour, procedural generation, input handling, and rendering systems. Each challenge describes the problem, the approach taken to resolve it, and the outcome of the solution.

Enemy AI Behaviour and Decision Consistency

One of the most complex systems to implement was the enemy combat AI. Early versions of the system used simple conditional logic, where enemies selected actions based on basic thresholds such as player health. While functional, this approach quickly became predictable and limited in terms of variation between enemy types.

The main issue was that behaviour lacked depth and often resulted in enemies repeating the same actions in similar situations. This reduced the strategic element of combat and made encounters feel repetitive over time.

To resolve this, the system was redesigned into a points-based evaluation model. Instead of using fixed conditions, each available action is assigned a weighted score based on the current combat state. Factors such as player health, enemy health and previous actions are all used to influence scoring. The action with the highest total score is then selected for execution.

This change significantly improved behaviour diversity, as enemies now respond dynamically to changing combat conditions rather than following fixed rules. However, it also introduced a new issue where enemies would consistently select the mathematically optimal action, which made behaviour feel too predictable in longer encounters.

To address this, a controlled randomness factor was added to the scoring system. This slightly modifies final scores within a defined range, ensuring that while enemies still behave logically, there is enough variation to prevent complete predictability. The result is a system that maintains strategic consistency while avoiding deterministic behaviour patterns.

Procedural Generation Reliability and Layout Stability

Another major technical challenge was the procedural temple generation system, which went through several iterations before reaching its final implementation. The initial prototype used a BSP (Binary Space Partitioning) approach, following guidance from supervision, combined with a tile-painting system where rooms were drawn onto a grid rather than instantiated as prefabs. While this approach was functional, it was difficult to control overall temple structure, and it often produced layouts where certain paths became excessively long or inefficient, negatively affecting the pacing and readability of exploration.

To address this, a second iteration moved to a more controlled grid-based generation system, still using tile-painting to construct rooms. This improved consistency and made the overall layout easier to structure and debug, but it still lacked modularity and limited the ability to create more varied and reusable room designs.

The final implementation evolved into a prefab-based room system. Instead of painting entire layouts, each room is now a self-contained prefab containing a main room grid alongside directional extension grids for each possible connection (north, south, east, and west). These directional sections represent both open and closed variants and are dynamically enabled or disabled depending on how the room connects to adjacent rooms.

This approach removes the need for separate corridor generation. Instead, connections between rooms are handled directly within each prefab, with the appropriate directional extensions activated to visually extend bridges between rooms. Since the temple is designed as a structure suspended over a void, these extensions function as the connecting paths between rooms.

This significantly improved control over navigation flow while still preserving procedural variation between temple runs. It also simplified the generation process, as room connectivity is handled internally within each prefab rather than requiring an additional corridor system.

The core challenge throughout all iterations was balancing randomness with readable and functional level design. Fully random approaches often resulted in poor pacing or inefficient traversal paths, while overly constrained systems reduced variation too heavily. The final system resolves this by generating a structured base layout first, then applying room logic through controlled placement and connection rules.

A validation step is also used to ensure reliability. If a generated temple does not meet minimum structural requirements, such as a valid number of rooms or properly formed progression paths, the entire layout is discarded and regenerated. While this introduces additional processing overhead, it ensures that every final dungeon is fully playable and maintains consistent structural quality.

Input System Conflicts and State Management

Managing player input across multiple gameplay states was another technical issue, particularly when transitioning between exploration, combat, and UI navigation. Early in development, input actions were handled within a single context, which caused conflicts when multiple systems were active at the same time. For example, movement inputs could still be registered while navigating menus, or combat inputs could interfere with UI interactions.

To resolve this, Unity's Input System was restructured using separate action maps for different game states. Exploration, combat, and UI each operate under their own input context, which is enabled or disabled depending on the current game state.

This change significantly improved system stability and made transitions between gameplay modes more predictable. It also simplified debugging, as input-related issues could be isolated to specific action maps rather than the entire system.

Dynamic Shader for Party-Based Highlighting

One unexpectedly challenging technical problem was implementing a dynamic shader used during combat to highlight party members. This shader creates a visual effect where the screen is darkened, except for circular areas centred on each active party member. It is used during specific interactions such as selecting targets for Ultimate Attacks or consumable item usage.

The challenge came from making the shader respond dynamically to the number and positions of party members. Since the party size can vary, the shader could not rely on a fixed number of highlight areas. Instead, it needed to support multiple circular masks that update in real time based on the current party configuration.

To solve this, the shader was designed to accept an array of positions representing each active party member, along with a corresponding radius for each highlight area. These values are updated from script during combat, allowing the shader to render multiple circular regions simultaneously. The rest of the screen is darkened using a mask-based approach, ensuring that only the selected areas remain visible.

One of the main difficulties was ensuring that the shader remained flexible without becoming overly complex. Handling multiple dynamic inputs required careful structuring of shader parameters and coordination between the gameplay code and rendering system. Additionally, performance needed to be considered to ensure that updating multiple mask regions each frame did not introduce noticeable overhead.

The final implementation successfully produces a responsive highlighting effect that adapts to different party sizes and combat situations. Beyond its visual purpose, it also improves clarity during key decision-making moments by directing player attention to relevant characters.

Summary of Technical Challenges

Overall, the main technical challenges in *The Mist* centred on refining systems through multiple iterations to balance flexibility, control, and consistency. The most significant challenges were in procedural generation and enemy AI, both of which required structural redesign rather than small adjustments.

Procedural generation required several iterations to achieve a balance between variation and navigational clarity. Early approaches produced layouts with inefficient or overly long traversal paths, which negatively affected pacing. The final system resolved this through a structured generation approach combined with prefab-based room design and validation checks.

Enemy AI also required refinement to achieve consistent but non-deterministic behaviour. The transition to a points-based decision system allowed enemies to react dynamically to combat conditions while maintaining variation through controlled randomness.

Across all systems, a consistent focus was placed on modular design and iterative testing. This made it possible to adjust individual systems without introducing instability elsewhere, ensuring that the final implementation remained both flexible and reliable.

Table 7: Technical Challenges Summary

Challenge	Issue	Solution	Outcome
Enemy AI behaviour	Repetitive or overly optimal decision-making	Points-based scoring system with controlled randomness	More adaptive and less predictable combat behaviour
Procedural generation	Inefficient layouts and overly long traversal paths	Iterative redesign (BSP → grid → prefab-based system with validation)	Improved pacing, readability, and reliable dungeon structure
Input system conflicts	Overlapping inputs between gameplay states	Separate Input Action Maps per state	Clear and stable transitions between exploration, combat, and UI
System interdependency	Changes in one system affecting others	Modular, data-driven design with continuous integration testing	Greater flexibility and reduced risk of breaking systems
Dynamic shader (combat selection)	Needed a visual system that highlights multiple party members dynamically based on party size	Custom shader using a mask with multiple position inputs to define visible areas around party members	Clear and responsive visual feedback during ultimate attacks and using consumables

7. Reflection & Learning

This section reflects on the development of *The Mist*, covering the skills I developed, key design decisions, challenges during production, and how my Semester 1 research translated into practice.

Technical and Programming Skills

One of the biggest areas of growth during this project was in programming and systems design within Unity. At the start of development, I had a decent understanding of how to build gameplay systems, but I quickly realised that creating a full game required a much more structured and scalable approach.

A key part of the project was building systems in a modular way. Systems such as enemy AI, procedural generation, and inventory management were built with a separation between data and logic, allowing for easy adjustment, debugging, and extension without impacting other systems. This modular structure ensured the project remained flexible and scalable throughout development.

Another key development area was working with Unity's Input System. Early on, I ran into issues where inputs would conflict when switching between gameplay states such as exploration, combat, and UI. The main improvement came from correctly enabling and disabling different input action maps depending on the current game state. This made the control flow much more reliable, as only the relevant inputs were active at any given time, preventing unintended actions and improving overall stability and responsiveness.

I also improved my understanding of debugging and iterative development. Instead of trying to build systems perfectly the first time, I started treating implementation as a process of refinement. This shift in mindset made development more manageable and reduced the pressure of getting everything correct immediately.

Design Lessons Learned

From a design perspective, one of the most important lessons was focusing on core gameplay before expanding systems. Early in development, I had a larger set of planned features, including puzzles, crafting, and expanded systems. By narrowing the focus to exploration, combat, and progression, I was able to create a more cohesive gameplay loop. It became clear that trying to include every planned feature would have spread the game too thin and reduced the quality of the core systems.

Another key lesson was the importance of clarity in system design. This was particularly noticeable in procedural generation and combat. Systems needed to be not only functional but also readable from a player perspective. For example, in dungeon generation, overly complex layouts created confusion during navigation, which led me to simplify the system into a more structured grid-based approach.

Similarly, in combat design, I learned that complexity needs to be balanced with readability. The enemy AI system initially risked becoming too deterministic, so introducing controlled randomness helped maintain strategic depth without making behaviour feel predictable.

Production and Time Management

Managing time effectively was one of the more challenging parts of the project. Although I used an iterative workflow, I initially underestimated how long system integration and debugging would take. Individual features often took longer than expected because they required adjustment once combined with other systems.

Using a Scrum-lite approach helped structure development into manageable cycles, but I found that flexibility was just as important as planning. Some systems, such as procedural generation and combat balancing, required multiple iterations before they felt stable, which meant I had to adjust sprint expectations frequently.

Trello helped keep tasks organised and made it easier to track progress across development. That said, I realised fairly quickly that real progress depended more on constant in-engine testing, which was where most issues became visible.

A key takeaway was that game development is not a linear process. Even when a feature seems finished, it often needs to be revisited once other systems are in place. Getting comfortable with that made the development process feel more manageable and less frustrating overall.

Semester 1 Research into Practice

One of the most useful parts of Semester 1 was researching procedural generation, AI behaviour models, and RPG progression systems. However, applying these ideas in practice highlighted the gap between understanding a system conceptually and implementing it in a working game.

For procedural generation, I initially explored more complex approaches including BSP-based dungeon generation, which was something I had been guided towards during early development discussions. I did build a working prototype using this method, along with a tile-based painting system rather than instantiating full room prefabs. While this helped me understand how procedural layouts could be constructed, I found it difficult to control overall dungeon flow, and it often led to layouts where paths became too long or awkward to navigate.

Because of this, I moved towards a simpler grid-based approach and eventually transitioned into a prefab-based room system. This gave me much more control over structure and allowed rooms to be designed as reusable components rather than being fully generated at a tile level. It also made it easier to maintain readable layouts and consistent pacing across temples.

A similar shift happened with AI. My initial understanding of enemy behaviour was fairly basic and relied on conditional logic tied to simple thresholds. Through development and testing, I refined this into a utility-based scoring system, where actions are evaluated based

on combat state and assigned weighted values. This produced more flexible and readable behaviour, while still allowing variation through controlled randomness.

Overall, Semester 1 provided the theoretical foundation, but the most important learning came from adapting and restructuring those ideas into systems that worked reliably within the constraints and goals of the game.

What I Would Do Differently

If I were starting the project again, one of the main changes I would make is spending more time on building and stabilising core systems before expanding the scope. In a few cases, I started adding features before the foundations were fully solid, which led to unnecessary rework later on.

I would also focus more on creating a vertical slice earlier in development. Having a smaller but fully functional section of the game early on would have made it easier to identify design issues and make better decisions before systems became more complex.

Another improvement would be tighter scope control from the beginning. I did end up reducing features later on, but making those decisions earlier would have helped keep more time available for refining the core gameplay experience.

Finally, I would have focused more on visual feedback and game feel earlier in development. While the systems themselves were solid, some aspects of presentation only really improved once I started iterating on them in the later stages.

Overall Reflection

Overall, *The Mist* was a valuable learning experience that pushed both my technical and design skills. It required thinking about systems as interconnected parts of a larger structure rather than isolated features, which changed how I approached development.

Although not all of the planned features made it into the final version, focusing on a smaller, more controlled scope resulted in a stronger core experience. The project helped me become more comfortable working iteratively and making practical decisions about what to prioritise during development.

It also gave me more confidence in building larger gameplay systems and managing a project where design, implementation, and iteration all need to stay aligned throughout development.

8. Future Work & Unimplemented Features

This section outlines features that were planned during early development of *The Mist* but didn't make it into the final build. Most of these were identified during pre-production or early sprint planning but were dropped later on to keep the project focused on finishing and polishing the core systems. In the end, the decision was mostly about scope rather than technical ability.

Crafting System and Harvestable Resources

Another planned system was resource gathering and crafting. The idea was to allow players to collect materials from the environment and use them to create consumables or upgrade items.

This was cut due to scope. While the inventory and item systems were already in place, adding a full resource loop would have required additional balancing, UI work, and integration with progression systems. It was ultimately decided that this would have added complexity without being essential to the core experience.

The underlying item framework is already flexible, so a crafting system could be introduced later with minimal structural changes.

Additional Enemy Types and Status Effects

Another planned area of expansion was more enemy variety and status effects, like poison or stun-style mechanics. These would have added more variety to combat encounters and allowed for more complex enemy setups.

This was mostly cut to avoid overcomplicating balancing. The current system already includes elemental weaknesses and a fairly flexible AI system, which gave enough depth for the scope of the project.

The way enemies are built is data-driven though, so adding more types or effects later would be straightforward if the system was extended.

Localisation and Language Support

A localisation system was also planned to support multiple languages. This would have included externalised text data and a system for switching language settings in-game.

This feature was not implemented due to time constraints and prioritisation of gameplay systems. While the game currently only supports a single language, the UI and dialogue systems were designed with external text handling in mind, so adding localisation later would be relatively straightforward.

Puzzle-Based Temple Progression

Originally, temples were going to include puzzle rooms alongside combat and treasure rooms. These would have been used to break up gameplay and add some variation to dungeon progression, likely involving switches, environmental interactions, or simple logic-based challenges.

This feature was cut mainly because of time. As development went on, it became clear that combat, procedural generation, and progression already needed more work than expected. Rather than trying to squeeze puzzles in at the last minute, I decided to focus on getting the core loop feeling solid and complete.

That said, this is still something that could be added later without too much trouble. The way the dungeon system is set up means puzzle rooms could just be another room type added into the existing generation process.

Expanded Skill Tree

The skill tree was originally intended to be larger, with more branching paths and specialised upgrades for different playstyles. Some of this exists in a limited form, but it was reduced in scope to focus on balancing core combat mechanics and ensuring progression felt consistent.

Since the system is modular and node-based, expanding it in future would mainly involve adding new nodes and tuning existing progression values rather than restructuring the system.

Expanded Quest System

The quest system was intended to include a wider range of main and side quests to provide more structured objectives across the game world. This would have helped reinforce narrative progression and encouraged more varied exploration outside of temple-based gameplay.

This was reduced in scope to prioritise completing and refining the core gameplay systems, particularly combat, procedural generation, and progression. However, the existing quest structure is already modular, meaning additional quests and branching objectives could be added in future without requiring changes to the underlying system.

Summary

Overall, the features that were not implemented were mainly removed due to scope control rather than technical issues. As the project evolved, it became more important to fully develop and refine the core systems instead of spreading development across additional mechanics.

Most of these features are still technically feasible within the current architecture. Systems like inventory, procedural generation, and progression were designed in a modular way, meaning they could be extended in future without major redesign.

The final scope reflects a deliberate decision to prioritise a complete, stable core experience over a wider but less polished feature set.

Table 8: Unimplemented Features

Summarises planned features that were not implemented within the final build of The Mist, along with their intended function and scope within the wider game design.

Feature	Description
Puzzle-Based Temple Progression	Puzzle rooms intended to appear within temples to add variation to dungeon progression, likely involving environmental interactions or simple logic challenges.
Crafting System & Harvestable Resources	System for collecting materials from the environment and using them to craft consumables or upgrade items.
Localisation / Language Support	Multi-language support system allowing text to be externalised and switched in-game.
Expanded Quest System	More main and side quests intended to deepen narrative structure and provide additional objectives beyond temple progression.
Larger Game World	Expansion of the overworld with more regions between temples, including additional exploration areas and content density.
Additional Temples & Elemental Powers (e.g. Wind)	More temple types introducing new elemental rewards beyond existing ones, such as wind-based abilities for traversal or combat variation.
Expanded Skill Tree	Larger skill tree with more branching paths and specialised upgrades for different playstyles.

All of the listed features are feasible within the current architecture of the game, as existing systems such as procedural generation, inventory, progression, and UI are designed in a modular way that would allow them to be extended or expanded without requiring major structural redesign.

9. Conclusion

The Mist was successfully developed as a 2D top-down RPG focused on turn-based combat, exploration, and procedural dungeon progression. The final product delivers a complete gameplay loop where the player explores a corrupted world, engages in strategic combat encounters, progresses through procedurally generated temples, and gradually strengthens their party through equipment, abilities, and progression systems.

Overall, the project met its original aim of creating a cohesive RPG experience built around interconnected systems. Core features such as combat, enemy AI, procedural generation, progression, and inventory management were fully implemented and integrated into a working gameplay loop. While some planned features were reduced in scope or removed during development, this was done deliberately to ensure that the core systems were stable, balanced, and fully playable rather than spreading development too thin across too many mechanics.

A key outcome of the project was the successful integration of multiple complex systems working together in real time. Combat is influenced by progression and equipment, enemy behaviour adapts dynamically through a utility-based AI system, and procedural generation provides varied but structured environments. This interconnected design helped reinforce the overall gameplay loop and maintain consistency across different parts of the game.

From a development perspective, one of the main takeaways was the importance of iteration and early system integration. Many of the systems evolved significantly through testing, particularly procedural generation and enemy AI, where initial versions were refined multiple times before reaching a stable and usable state. Adopting a flexible, iterative workflow made it possible to identify issues early and adjust direction when necessary without disrupting overall progress.

Another key lesson was the importance of scope management. Several planned features were reduced or removed to ensure the project remained focused on delivering a complete core experience. This decision ultimately improved the final quality of the game by allowing more time to be spent refining the systems that mattered most.

In conclusion, the project demonstrates a functional and cohesive RPG built through iterative development, system interdependency, and structured problem-solving, resulting in a complete and playable final product.

10. Appendices

Development logs

UI system: https://youtu.be/MIUxM_JnZgc?si=oOytKPqMK4IgtuUH

Combat system: <https://youtu.be/7OMbSX-Eoew?si=kQ-u4L5JKkWUU8U2>

Prototype hamster gameplay: <https://youtu.be/kS1GoOk8fc0?si=jgxWEoCbazirON0J>

Project management

Miro board: https://miro.com/app/board/uXjVGaqySc0=/?share_link_id=580089923835

Trello board: <https://trello.com/b/p6oUWoYQ/fyp>

Executable download

Itch.io: <https://hillyleopard133.itch.io/the-mist>

11. Credits

Audio

Original Music: Ashley Holloway

Assets

Itch.io

- Temple interior tileset: Penusbmic
- Temple exterior tileset: ELV Games
- Temple relics and bosses: Penusbmic
- Rat boss: Beowulf

Unity asset store

- Game input icons: Amanz
- UX icons: Heathen Engineering
- Cave tileset: Szadi Art

Tools

JSON serializer: BayatGames